

Compiladores

Analísadores léxicos

REALIZAR APÓS posição 20: Aula 18 - Exercícios e Revisão | RegExp | Diagramas de Transição | Analísador Léxico | Compiladores

Posições na playlist 14 a 20 | 120 minutos | 100 pontos

Curso principal: Compiladores |

<https://youtube.com/playlist?list=PLX6Nyaq0ebfhI396WIWN6WIBm-tp7vDtV&si=LoaU9IzLMuSVikgi>

Conteúdo verificável: Regex; tokens; transições; Flex; aplicações; AFN/AFD; geração; minimização

Instruções

Resolva sem consultar o gabarito. Apresente desenvolvimento, hipóteses, unidades e verificações. Código pode ser escrito em papel ou editor sem sugestões automáticas; use somente recursos compatíveis com o bloco. Ferramentas de simulação são permitidas apenas onde solicitadas. Use folhas adicionais para diagramas e cálculos.

Pontuação: Q1 = 15; Q2 = 25; Q3 = 25; Q4 = 35. Crédito parcial depende de raciocínio demonstrado. Meta: ≥ 80 prosseguir; 70–79 revisar erros; < 70 revisar o bloco.

Aluno(a): _____ Data: _____

Alinhamento: títulos/ordem e informações públicas do curso. As aulas não foram integralmente assistidas; detalhes não expostos não estão certificados.

Padrões e prioridade

Defina regex para identificadores que começam por letra e continuam por letras/dígitos, e inteiros com ao menos um dígito. Tokenize 'if ifx 12'. Use maior lexema e prioridade de palavra-chave no empate. Explique por que reconhecer 'if' no prefixo de 'ifx' seria incorreto.

Desenvolvimento / justificativa / esquema:

AFD de números

Construa AFD para números com um ou mais dígitos e parte decimal opcional, que exige ao menos um dígito depois do ponto. Dê tabela de transição incluindo estado morto, marque iniciais/finais e teste 12, 12.3, 12., .3. Explique como o lexer pode aceitar um prefixo sem aceitar toda a cadeia.

Desenvolvimento / justificativa / esquema:

Flex aplicado

Escreva regras Flex para contar inteiros, identificadores e quebras de linha, ignorar espaços e reportar caracteres desconhecidos. Explique yytext/yylen e o que fazer se houver regra especial para 'if'. Não precisa criar programa completo de integração.

Desenvolvimento / justificativa / esquema:

Subconjuntos e minimização

AFN tem q_0 inicial com laço a e transição a para q_1 ; q_1 tem b para q_2 final. Converta por subconjuntos para AFD, dê tabela para a , b e linguagem reconhecida. Determine se estados finais e não finais podem ser fundidos e justifique distinguindo cadeias.

Desenvolvimento / justificativa / esquema:

Gabarito e Resoluções

Abra esta seção somente depois de concluir a tentativa. Há mais de um método correto possível; compare também o processo e os casos extremos.

Critérios de correção

Q1: 4 pontos pela modelagem/conceito, 8 pela solução, 3 pela verificação. Q2 e Q3: 6 pela modelagem, 14 pela solução, 5 pela justificativa/verificação. Q4: 8 pela modelagem, 20 pela solução, 7 pela análise crítica e casos extremos. Nos itens dissertativos, a solução inclui os conceitos e relações obrigatórios; crédito integral exige conectar esses conceitos ao cenário. Em código, valorize correção do contrato, casos base e explicação; uma chamada a biblioteca que evita a implementação pedida não substitui o algoritmo.

Questão 1 - Padrões e prioridade (15 pontos)

ID=[A-Za-z][A-Za-z0-9]*, INT=[0-9]+. Saída IF, ID(ifx), INT(12), ignorando espaços. 'ifx' tem lexema maior que 'if'; prioridade só resolve empate de comprimento, portanto não divide ifx em IF e ID. Palavras-chave podem ser regras anteriores ou checagem em tabela após reconhecer identificador.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 2 - AFD de números (25 pontos)

Estados q0 inicial, q1 inteiro final, q2 após ponto não final, q3 decimal final, qM morto.
q0:dígito→q1; q1:dígito→q1, ponto→q2; q2:dígito→q3; q3:dígito→q3; demais→qM, que fica em si. 12 e 12.3 aceitos; 12. e 3 não são números completos nesse padrão. Lexer maximal munch guarda último estado final: pode reconhecer 12 e depois tratar ponto separadamente se linguagem o permite. Reconhecer cadeia inteira e extrair token são contratos distintos.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 3 - Flex aplicado (25 pontos)

Regras possíveis:

```
[0-9]+      { numeros++; }
[A-Za-z][A-Za-z0-9]* { identificadores++; }
\n         { linhas++; }
[ \t]+     { /* ignorar */ }
.         { reportar(yytext); }
```

yytext contém lexema corrente; yyleng seu comprimento. Regra 'if' antes de ID vence empate; ifx continua ID pela maior extensão. Contagem de linhas deve definir inicialização/última linha sem newline. Integração pode tratar EOF/arquivos múltiplos, mas não é exigida na regra básica.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 4 - Subconjuntos e minimização (35 pontos)

Estados alcançáveis $A = \{q_0\}$, $B = \{q_0, q_1\}$, $C = \{q_2\}$, $D = \emptyset$. $A: a \rightarrow B, b \rightarrow D$; $B: a \rightarrow B, b \rightarrow C$; $C: a/b \rightarrow D$; $D: a/b \rightarrow D$. Só C final. Linguagem a^+b . C distingue-se de não finais por ϵ ; A e B por b (só B aceita); D distingue-se de A por ab e de B por b . Logo todos distintos, AFD mínimo de 4 estados. Incluir morto completa função de transição.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Diagnóstico do checkpoint

Registre sua nota e os conceitos a revisar em progresso.csv. Antes de avançar, refaça os itens errados sem olhar a resolução e crie um pequeno exemplo novo que use a mesma ideia. Nota alta sem justificar respostas não demonstra domínio.